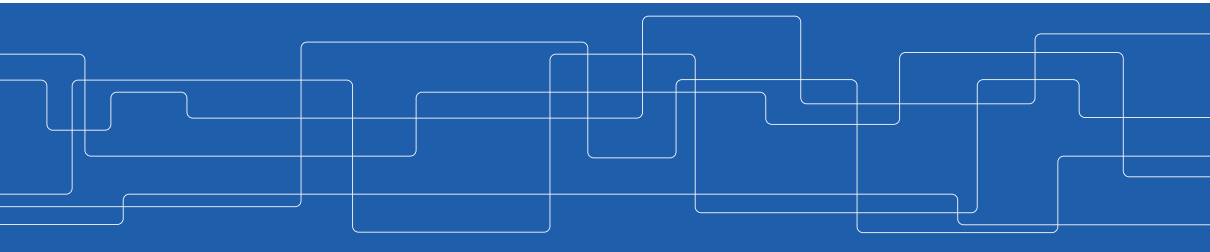


Convolutional Neural Networks

Amir H. Payberah
payberah@kth.se
2026-11-09





Let's Start With An Example

MNIST Dataset

- Handwritten digits in the MNIST dataset are 28x28 pixel greyscale images.



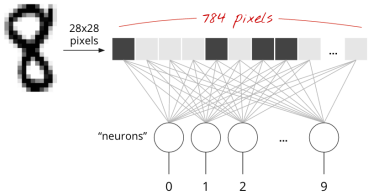
One-Layer Network For Classifying MNIST (1/4)



[https://github.com/GoogleCloudPlatform/tensorflow-without-a-phd]

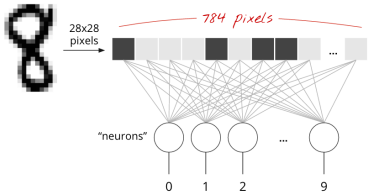
One-Layer Network For Classifying MNIST (2/4)

- Let's make a one-layer neural network for classifying digits.



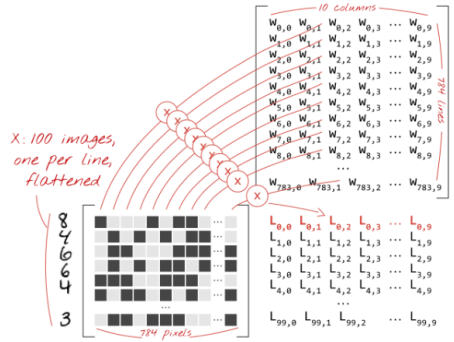
One-Layer Network For Classifying MNIST (2/4)

- Let's make a one-layer neural network for classifying digits.
- Each neuron in a neural network:
  - Does a weighted sum of all of its inputs
  - Adds a bias
  - Feeds the result through some non-linear activation function, e.g., softmax.



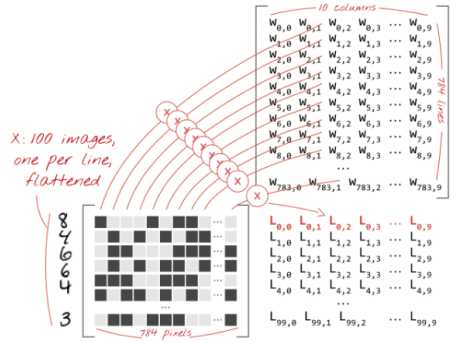
One-Layer Network For Classifying MNIST (3/4)

- Assume we have a batch of 100 images as the input.



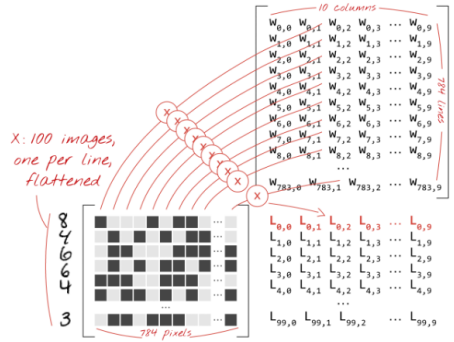
One-Layer Network For Classifying MNIST (3/4)

- Assume we have a batch of 100 images as the input.
- Using the first column of the weights matrix **W**, we compute the weighted sum of all the pixels of the first image.



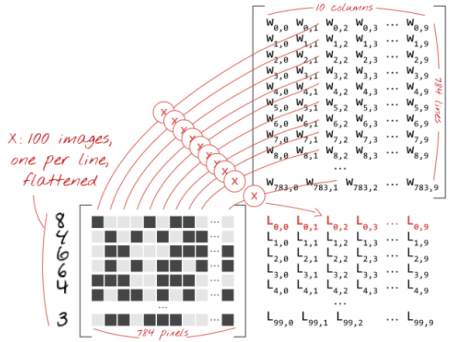
One-Layer Network For Classifying MNIST (3/4)

- Assume we have a batch of 100 images as the input.
- Using the first column of the weights matrix **W**, we compute the weighted sum of all the pixels of the first image.
  - The first neuron:
    $L_{0,0} = w_{0,0}x_0^{(1)} + w_{1,0}x_1^{(1)} + \cdots + w_{783,0}x_{783}^{(1)}$



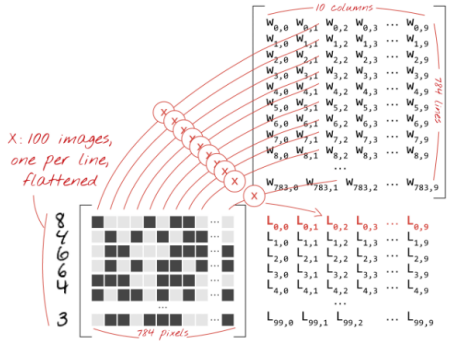
One-Layer Network For Classifying MNIST (3/4)

- Assume we have a batch of 100 images as the input.
- Using the first column of the weights matrix **W**, we compute the weighted sum of all the pixels of the first image.
  - The first neuron:
    $L_{0,0} = w_{0,0}x_0^{(1)} + w_{1,0}x_1^{(1)} + \cdots + w_{783,0}x_{783}^{(1)}$
  - The 2nd neuron until the 10th:
    $L_{0,1} = w_{0,1}x_0^{(1)} + w_{1,1}x_1^{(1)} + \cdots + w_{783,1}x_{783}^{(1)}$
    $\cdots$
    $L_{0,9} = w_{0,9}x_0^{(1)} + w_{1,9}x_1^{(1)} + \cdots + w_{783,9}x_{783}^{(1)}$



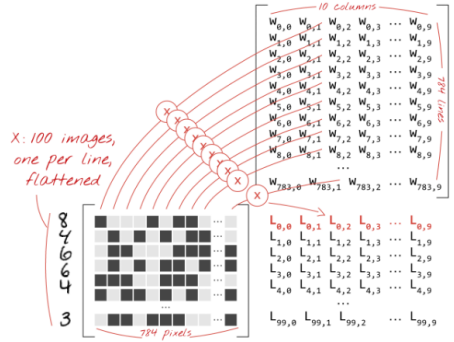
One-Layer Network For Classifying MNIST (3/4)

- Assume we have a batch of 100 images as the input.
- Using the first column of the weights matrix **W**, we compute the weighted sum of all the pixels of the first image.
  - The first neuron:
    $L_{0,0} = w_{0,0}x_0^{(1)} + w_{1,0}x_1^{(1)} + \cdots + w_{783,0}x_{783}^{(1)}$
  - The 2nd neuron until the 10th:
    $L_{0,1} = w_{0,1}x_0^{(1)} + w_{1,1}x_1^{(1)} + \cdots + w_{783,1}x_{783}^{(1)}$
    $\cdots$
    $L_{0,9} = w_{0,9}x_0^{(1)} + w_{1,9}x_1^{(1)} + \cdots + w_{783,9}x_{783}^{(1)}$
  - Repeat the operation for the other 99 images, i.e., $\mathbf{x}^{(2)} \cdots \mathbf{x}^{(100)}$



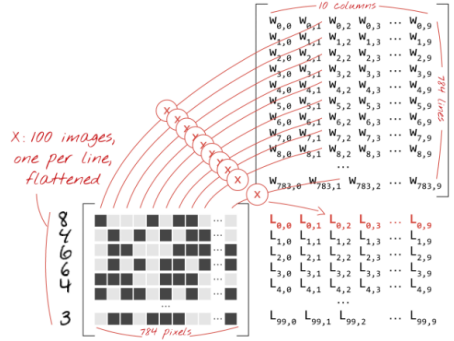
One-Layer Network For Classifying MNIST (4/4)

- Each neuron must now add its bias.
- Apply the softmax activation function for each instance $\mathbf{x}^{(i)}$.
- For each input instance $\mathbf{x}^{(i)}$: $\mathbf{L_i} = \begin{bmatrix} L_{i,0} \\ L_{i,1} \\ \vdots \\ L_{i,9} \end{bmatrix}$



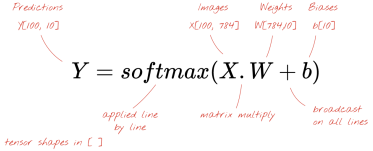
One-Layer Network For Classifying MNIST (4/4)

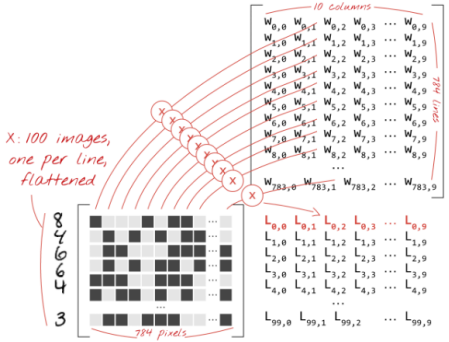
- Each neuron must now add its bias.
- Apply the softmax activation function for each instance $\mathbf{x}^{(i)}$.
- For each input instance $\mathbf{x}^{(i)}$: $\mathbf{L_i} = \begin{bmatrix} L_{i,0} \\ L_{i,1} \\ \vdots \\ L_{i,9} \end{bmatrix}$
- $\hat{\mathbf{y}}_i = \texttt{softmax}(\mathbf{L_i} + \mathbf{b})$



One-Layer Network For Classifying MNIST (4/4)

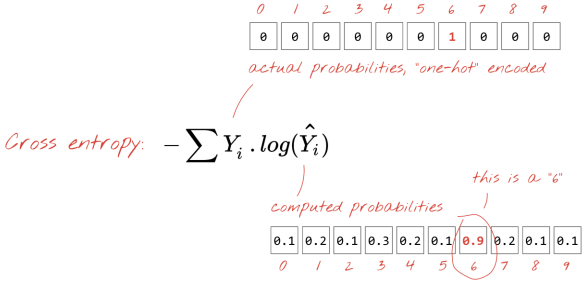
- Each neuron must now add its bias.
- Apply the softmax activation function for each instance $\mathbf{x}^{(i)}$.
- For each input instance $\mathbf{x}^{(i)}$: $\mathbf{L_i} = \begin{bmatrix} L_{i,0} \\ L_{i,1} \\ \vdots \\ L_{i,9} \end{bmatrix}$
- $\hat{\mathbf{y}}_i = \texttt{softmax}(\mathbf{L_i} + \mathbf{b})$





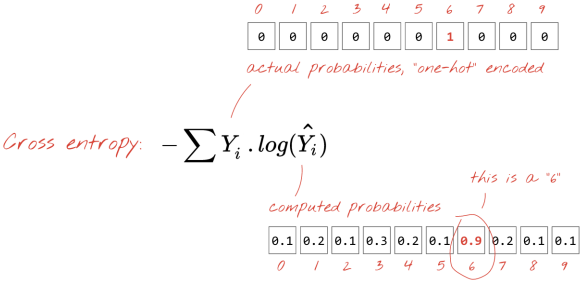
How Good the Predictions Are?

- Define the cost function $J(\mathbf{W})$ as the cross-entropy of what the network tells us ($\hat{\mathbf{y}}_i$) and what we know to be the truth ($\mathbf{y}_i$), for each instance $\mathbf{x}^{(i)}$.



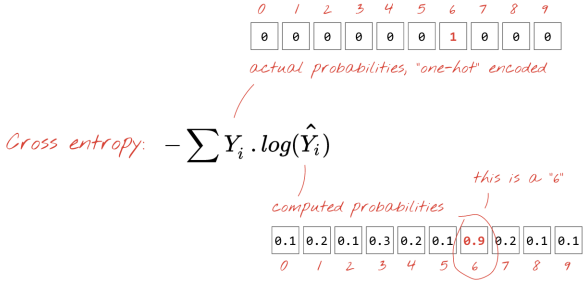
How Good the Predictions Are?

- Define the cost function $J(\mathbf{W})$ as the cross-entropy of what the network tells us ($\hat{\mathbf{y}}_i$) and what we know to be the truth ($\mathbf{y}_i$), for each instance $\mathbf{x}^{(i)}$.
- Compute the partial derivatives of the cross-entropy with respect to all the weights and all the biases, $\nabla_{\mathbf{W}} J(\mathbf{W})$.



How Good the Predictions Are?

- Define the cost function $\mathtt{J}(\mathbf{W})$ as the cross-entropy of what the network tells us ($\hat{\mathbf{y}}_\mathtt{i}$) and what we know to be the truth ($\mathbf{y}_\mathtt{i}$), for each instance $\mathbf{x}^{(\mathtt{i})}$.
- Compute the partial derivatives of the cross-entropy with respect to all the weights and all the biases, $\nabla_{\mathbf{W}}\mathtt{J}(\mathbf{W})$.
- Update weights and biases by a fraction of the gradient $\mathbf{W}^{(\mathtt{next})} = \mathbf{W} - \eta\nabla_{\mathbf{W}}\mathtt{J}(\mathbf{W})$





```
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```




```
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
model = tf.keras.Sequential([
  tf.keras.layers.Flatten(input_shape=(28, 28)),
  tf.keras.layers.Dense(10, activation='softmax')
])
```




```
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
model = tf.keras.Sequential([
  tf.keras.layers.Flatten(input_shape=(28, 28)),
  tf.keras.layers.Dense(10, activation='softmax')
])
```

```
model.compile(optimizer='sgd', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

```
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
model = tf.keras.Sequential([
  tf.keras.layers.Flatten(input_shape=(28, 28)),
  tf.keras.layers.Dense(10, activation='softmax')
])
```

```
model.compile(optimizer='sgd', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

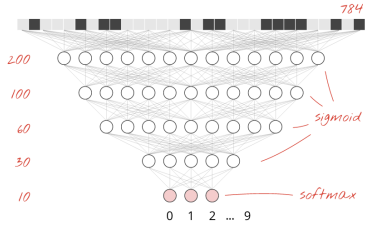
```
model.fit(x_train, y_train, batch_size=100, epochs=10)
model.evaluate(x_test,  y_test)
```


CONTINUOUS
IMPROVEMENT

Some Improvement (1/5)

- Add more layers to improve the accuracy.
- On intermediate layers we will use the the sigmoid activation function.
- We keep softmax as the activation function on the last layer.





[https://github.com/GoogleCloudPlatform/tensorflow-without-a-phd]

Some Improvement (2/5)

- Network initialization, e.g., using He initialization.
- Better optimizer, e.g., using Adam optimizer.

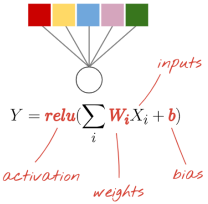


[https://github.com/GoogleCloudPlatform/tensorflow-without-a-phd]

Some Improvement (3/5)

- Better activation function, e.g., using $\texttt{ReLU}(z) = \max(0, z)$.



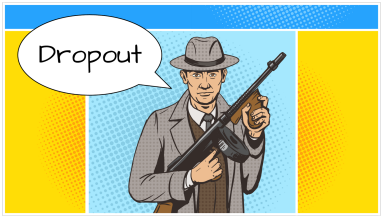


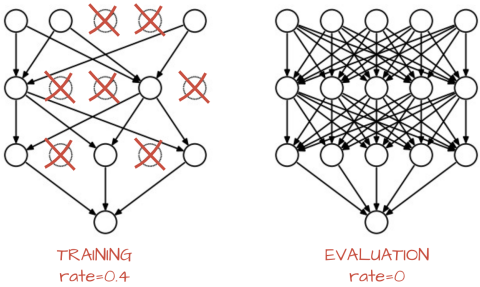


[https://github.com/GoogleCloudPlatform/tensorflow-without-a-phd]

Some Improvement (4/5)

- Overcome overfitting, e.g., using dropout.





[https://github.com/GoogleCloudPlatform/tensorflow-without-a-phd]

Some Improvement (5/5)

- Start fast and decay the learning rate exponentially.
- You can do this with the `tf.keras.callbacks.LearningRateScheduler` callback.



[https://github.com/GoogleCloudPlatform/tensorflow-without-a-phd]

```
model = tf.keras.models.Sequential([
  tf.keras.layers.Flatten(input_shape=(28, 28)),
  tf.keras.layers.Dense(128, kernel_initializer="he_normal", activation='relu'),
  tf.keras.layers.Dropout(0.2),
  tf.keras.layers.Dense(10, activation='softmax')
])
```

```
model = tf.keras.models.Sequential([
  tf.keras.layers.Flatten(input_shape=(28, 28)),
  tf.keras.layers.Dense(128, kernel_initializer="he_normal", activation='relu'),
  tf.keras.layers.Dropout(0.2),
  tf.keras.layers.Dense(10, activation='softmax')
])
```

```
# lr decay function
def lr_decay(epoch):
    return 0.01 * (0.6 ** epoch)

# lr schedule callback
lr_decay_callback = tf.keras.callbacks.LearningRateScheduler(lr_decay, verbose=True)

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

```
model = tf.keras.models.Sequential([
  tf.keras.layers.Flatten(input_shape=(28, 28)),
  tf.keras.layers.Dense(128, kernel_initializer="he_normal", activation='relu'),
  tf.keras.layers.Dropout(0.2),
  tf.keras.layers.Dense(10, activation='softmax')
])
```

```
# lr decay function
def lr_decay(epoch):
    return 0.01 * (0.6 ** epoch)

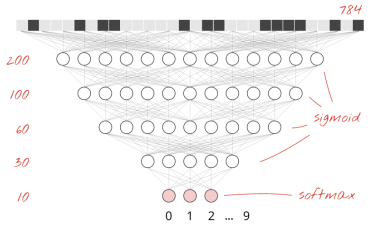
# lr schedule callback
lr_decay_callback = tf.keras.callbacks.LearningRateScheduler(lr_decay, verbose=True)

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

```
model.fit(x_train, y_train, batch_size=100, epochs=10, callbacks=[lr_decay_callback])
model.evaluate(x_test,  y_test)
```

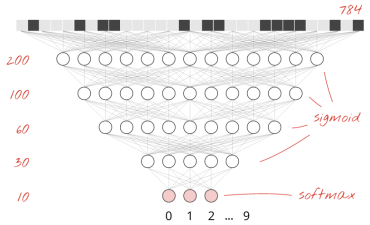
Vanilla Deep Neural Networks Challenges (1/2)

- Pixels of each image were flattened into a single vector (really bad idea).



Vanilla Deep Neural Networks Challenges (1/2)

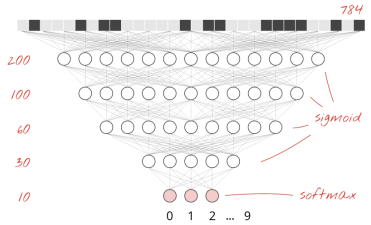
- Pixels of each image were flattened into a single vector (really bad idea).



- Vanilla deep neural networks do not scale.
  - In MNIST, images are black-and-white 28x28 pixel images: $28 \times 28 = 784$ weights.

Vanilla Deep Neural Networks Challenges (1/2)

- Pixels of each image were flattened into a single vector (really bad idea).



- Vanilla deep neural networks do not scale.
  - In MNIST, images are black-and-white 28x28 pixel images: $28 \times 28 = 784$ weights.
- Handwritten digits are made of shapes and we discarded the shape information when we flattened the pixels.



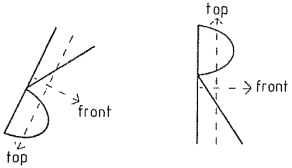
Vanilla Deep Neural Networks Challenges (2/2)

- Difficult to recognize objects.

Vanilla Deep Neural Networks Challenges (2/2)

- Difficult to recognize objects.
- Rotation
- Lighting: objects may look different depending on the level of external lighting.
- Deformation: objects can be deformed in a variety of non-affine ways.
- Scale variation: visual classes often exhibit variation in their size.
- Viewpoint invariance.







Tackle the Challenges

- Convolutional neural networks (CNN) can tackle the vanilla model challenges.
- CNN is a type of neural network that can take advantage of shape information.



Tackle the Challenges

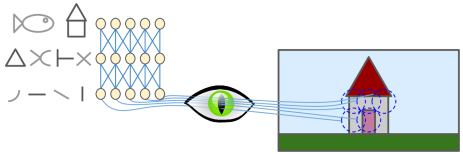
- Convolutional neural networks (CNN) can tackle the vanilla model challenges.
- CNN is a type of neural network that can take advantage of shape information.
- It applies a series of filters to the raw pixel data of an image to extract and learn higher-level features, which the model can then use for classification.

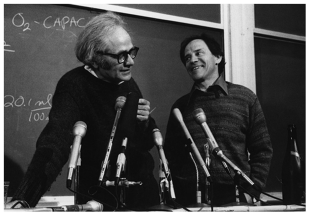


Filters and Convolution Operations

Brain Visual Cortex Inspired CNNs

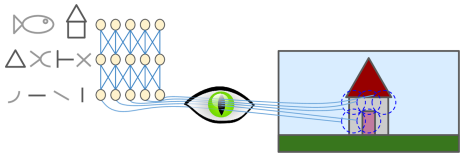
- 1959, David H. Hubel and Torsten Wiesel.
- Many neurons in the visual cortex have a small local receptive field.

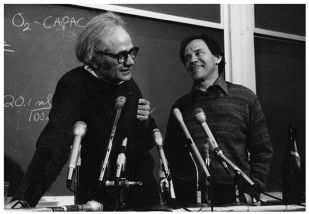




Brain Visual Cortex Inspired CNNs

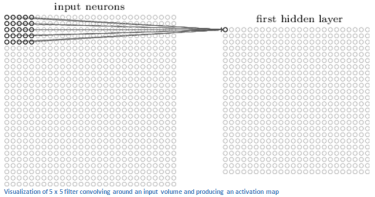
- 1959, David H. Hubel and Torsten Wiesel.
- Many neurons in the visual cortex have a small local receptive field.
- They react only to visual stimuli located in a limited region of the visual field.





Receptive Fields and Filters

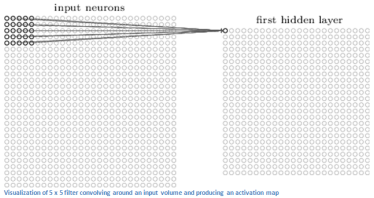
- Imagine a flashlight that is shining over the top left of the image.



[https://adeshpande3.github.io/A-Beginner's-Guide-To-Understanding-Convolutional-Neural-Networks]

Receptive Fields and Filters

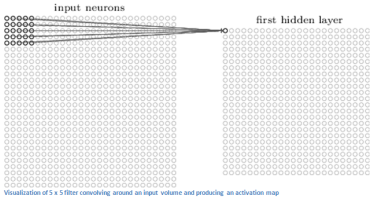
- Imagine a flashlight that is shining over the top left of the image.
- The region that it is shining over is called the receptive field.
- This flashlight is called a filter.



[https://adeshpande3.github.io/A-Beginner's-Guide-To-Understanding-Convolutional-Neural-Networks]

Receptive Fields and Filters

- Imagine a flashlight that is shining over the top left of the image.
- The region that it is shining over is called the receptive field.
- This flashlight is called a filter.
- A filter is a set of weights.
- A filter is a feature detector, e.g., straight edges, simple colors, and curves.

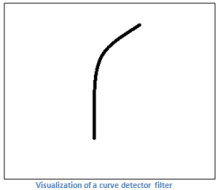


[https://adeshpande3.github.io/A-Beginner's-Guide-To-Understanding-Convolutional-Neural-Networks]

Filters Example (1/3)

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter

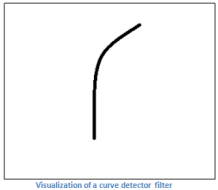


Visualization of a curve detector filter

Filters Example (1/3)

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

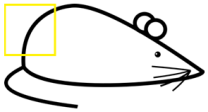
Pixel representation of filter



Visualization of a curve detector filter



Original image



Visualization of the filter on the image

[https://adeshpande3.github.io/A-Beginner's-Guide-To-Understanding-Convolutional-Neural-Networks]

Filters Example (2/3)



Visualization of the receptive field

0	0	0	0	0	0	30
0	0	0	0	50	50	50
0	0	0	20	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0

Pixel representation of the receptive field

*

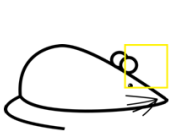
0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter

Multiplication and Summation = (50*30)+(50*30)+(50*30)+(20*30)+(50*30) = 6600 (A large number!)

[https://adeshpande3.github.io/A-Beginner's-Guide-To-Understanding-Convolutional-Neural-Networks]

Filters Example (3/3)



Visualization of the filter on the image

0	0	0	0	0	0	0
0	40	0	0	0	0	0
40	0	40	0	0	0	0
40	20	0	0	0	0	0
0	50	0	0	0	0	0
0	0	50	0	0	0	0
25	25	0	50	0	0	0

Pixel representation of receptive field

$*$

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

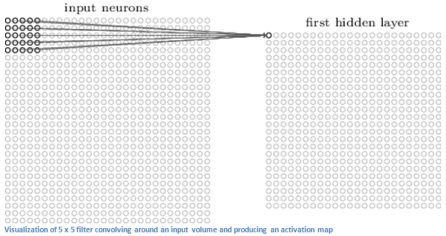
Pixel representation of filter

Multiplication and Summation = 0

[https://adeshpande3.github.io/A-Beginner's-Guide-To-Understanding-Convolutional-Neural-Networks]

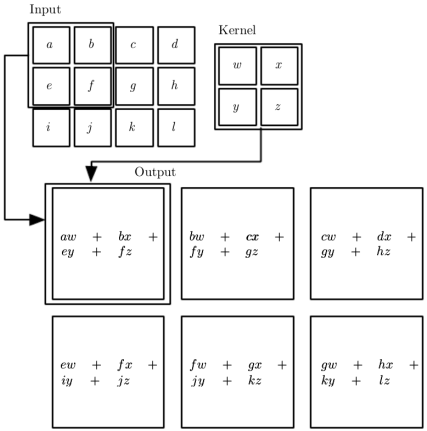
Convolution Operation

- Convolution takes a filter and multiplying it over the entire area of an input image.
- Imagine this flashlight (filter) sliding across all the areas of the input image.



[https://adeshpande3.github.io/A-Beginner's-Guide-To-Understanding-Convolutional-Neural-Networks]

Convolution Operation - 2D Example

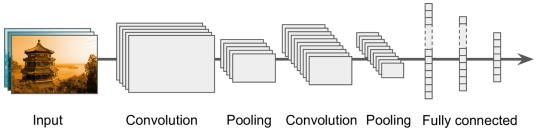




Convolutional Neural Network (CNN)

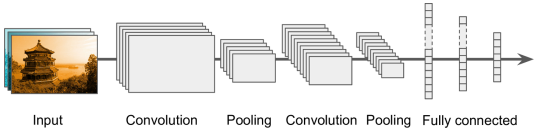
CNN Components (1/2)

- Convolutional layers: apply a specified number of convolution filters to the image.



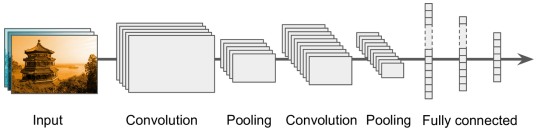
CNN Components (1/2)

- Convolutional layers: apply a specified number of convolution filters to the image.
- Pooling layers: downsample the image data extracted by the convolutional layers to reduce the dimensionality of the feature map in order to decrease processing time.



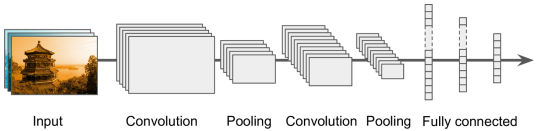
CNN Components (1/2)

- Convolutional layers: apply a specified number of convolution filters to the image.
- Pooling layers: downsample the image data extracted by the convolutional layers to reduce the dimensionality of the feature map in order to decrease processing time.
- Dense layers: a fully connected layer that performs classification on the features extracted by the convolutional layers and downsampled by the pooling layers.



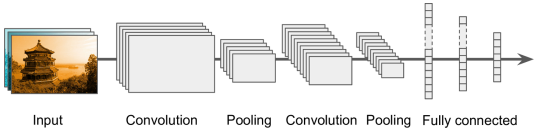
CNN Components (2/2)

- A CNN is composed of a stack of convolutional modules.



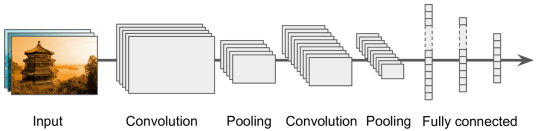
CNN Components (2/2)

- A CNN is composed of a stack of convolutional modules.
- Each module consists of a convolutional layer followed by a pooling layer.



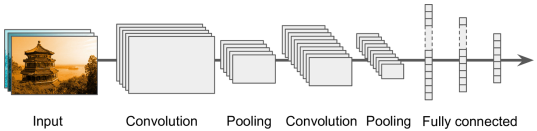
CNN Components (2/2)

- A CNN is composed of a stack of convolutional modules.
- Each module consists of a convolutional layer followed by a pooling layer.
- The last module is followed by one or more dense layers that perform classification.

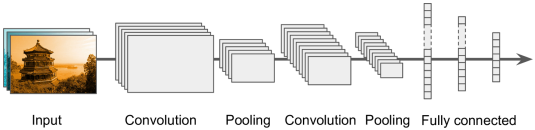


CNN Components (2/2)

- A CNN is composed of a stack of convolutional modules.
- Each module consists of a convolutional layer followed by a pooling layer.
- The last module is followed by one or more dense layers that perform classification.
- The final dense layer contains a single node for each target class in the model, with a softmax activation function.

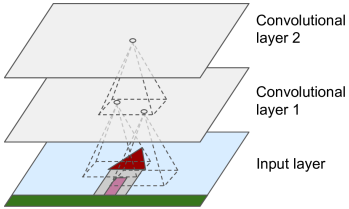


Convolutional Layer



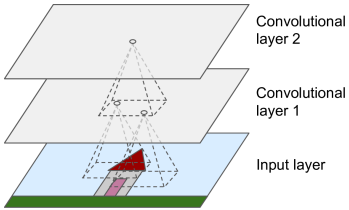
Convolutional Layer

- Each neuron in the convolutional layers is only connected to pixels in its receptive field (not every single pixel).
- Each neuron applies filters on its receptive field.



Convolutional Layer

- Each neuron in the convolutional layers is only connected to pixels in its receptive field (not every single pixel).
- Each neuron applies filters on its receptive field.
- All neurons of a convolutional layer reuse the same weights.



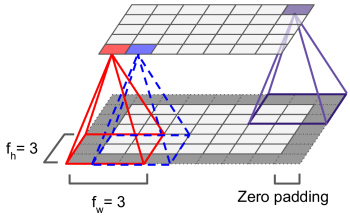


Padding

- What will happen if you apply a 5x5 filter to a 32x32 input volume?
  - The output volume would be 28x28.
  - The spatial dimensions decrease.

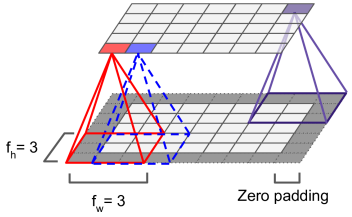
Padding

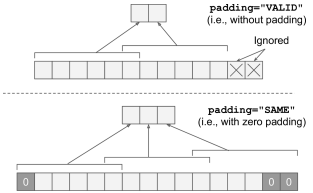
- What will happen if you apply a 5x5 filter to a 32x32 input volume?
  - The output volume would be 28x28.
  - The spatial dimensions decrease.
- Zero padding: in order for a layer to have the same height and width as the previous layer, it is common to add zeros around the inputs.



Padding

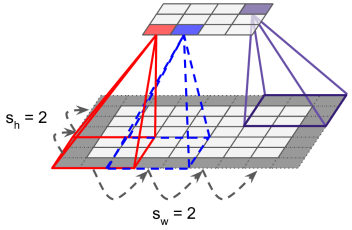
- What will happen if you apply a 5x5 filter to a 32x32 input volume?
  - The output volume would be 28x28.
  - The spatial dimensions decrease.
- Zero padding: in order for a layer to have the same height and width as the previous layer, it is common to add zeros around the inputs.
- In TensorFlow, padding can be either `SAME` or `VALID` to have zero padding or not.





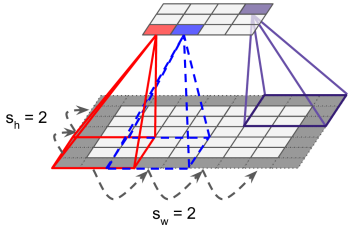
Stride

- The distance between two consecutive receptive fields is called the stride.



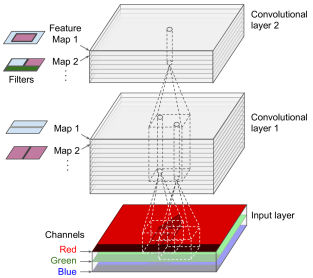
Stride

- The distance between two consecutive receptive fields is called the stride.
- The stride controls how the filter convolves around the input volume.



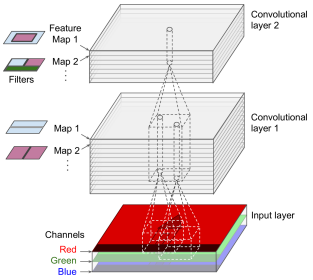
Stacking Multiple Feature Maps

- Up to now, we represented each convolutional layer with a single feature map.



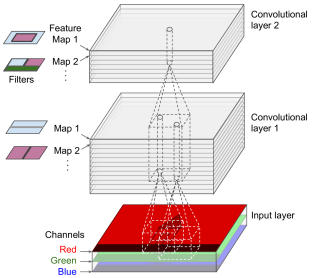
Stacking Multiple Feature Maps

- Up to now, we represented each convolutional layer with a single feature map.
- Each convolutional layer can be composed of several feature maps of equal sizes.

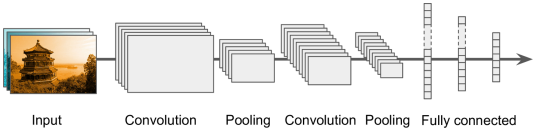


Stacking Multiple Feature Maps

- Up to now, we represented each convolutional layer with a single feature map.
- Each convolutional layer can be composed of several feature maps of equal sizes.
- A convolutional layer simultaneously applies multiple filters to its inputs.

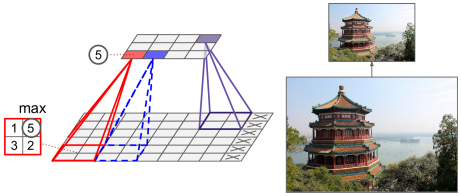


Pooling Layer



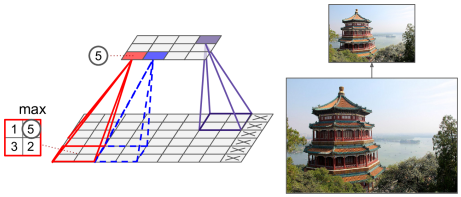
Pooling Layer (1/2)

- After the activation functions, we can apply a pooling layer.
- Its goal is to subsample (shrink) the input image.



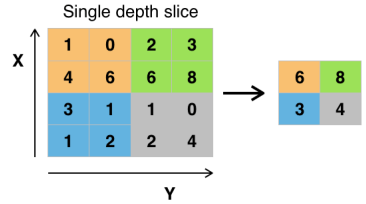
Pooling Layer (1/2)

- After the activation functions, we can apply a pooling layer.
- Its goal is to subsample (shrink) the input image.
  - To reduce the computational load, the memory usage, and the number of parameters.



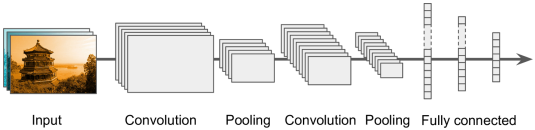
Pooling Layer (2/2)

- Each neuron in a pooling layer is connected to the outputs of a receptive field in the previous layer.
- A pooling neuron has no weights.
- It aggregates the inputs using an aggregation function such as the max or mean.



Example of Maxpool with a 2x2 filter and a stride of 2

Fully Connected Layer





Fully Connected Layer

- This layer takes an input from the last convolution module, and outputs an `N` dimensional vector.
  - `N` is the number of classes that the model has to choose from.



Fully Connected Layer

- This layer takes an input from the last convolution module, and outputs an `N` dimensional vector.
  - `N` is the number of classes that the model has to choose from.
- For example, if you wanted a digit classification model, `N` would be 10.



Fully Connected Layer

- This layer takes an input from the last convolution module, and outputs an `N` dimensional vector.
  - `N` is the number of classes that the model has to choose from.
- For example, if you wanted a digit classification model, `N` would be 10.
- Each number in this `N` dimensional vector represents the probability of a certain class.



Flattening

- We need to convert the output of the convolutional part of the CNN into a 1D feature vector.
- This operation is called flattening.



Flattening

- We need to convert the output of the convolutional part of the CNN into a 1D feature vector.
- This operation is called flattening.
- It gets the output of the convolutional layers, flattens all its structure to create a single long feature vector to be used by the dense layer for the final classification.

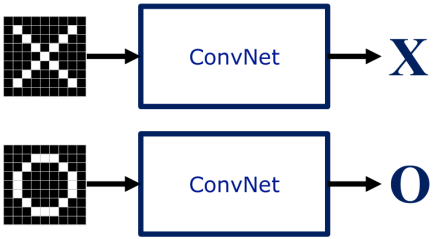


Example

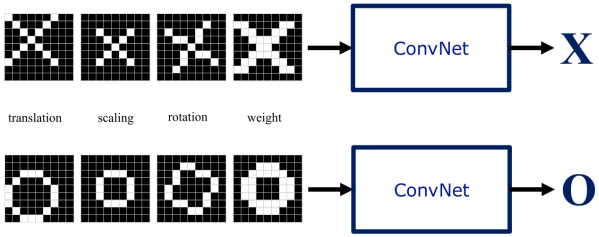
A Toy ConvNet: X's and O's



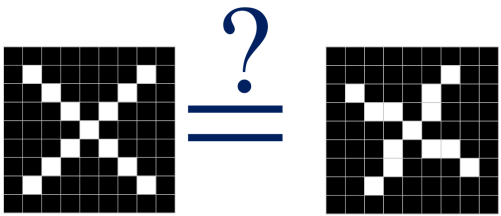
For Example



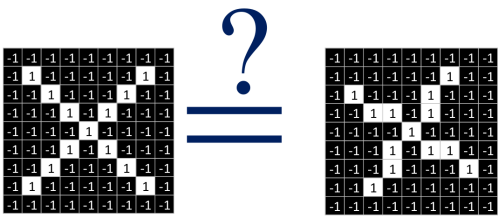
Trickier Cases



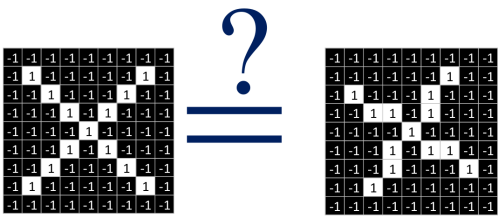
Deciding is Hard

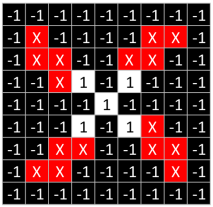


What Computers See

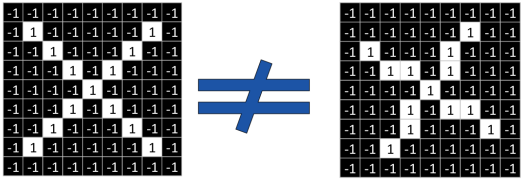


What Computers See

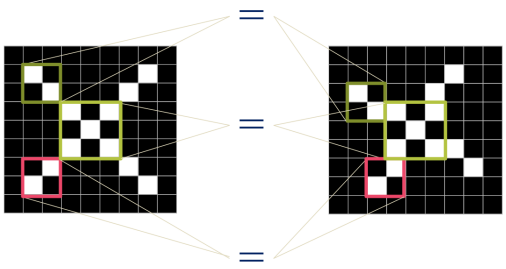




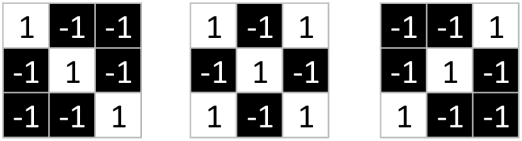
Computers are Literal



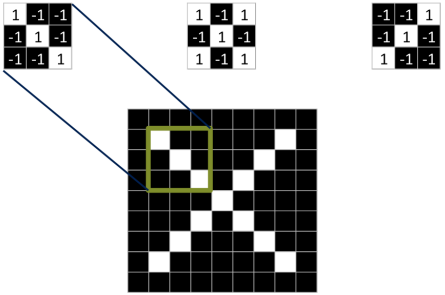
ConvNets Match Pieces of the Image



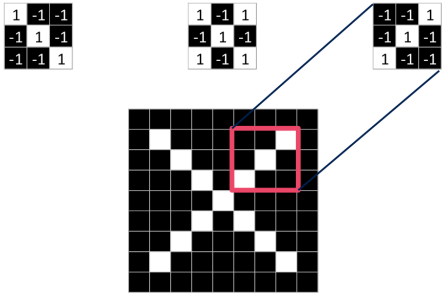
Filters Match Pieces of the Image



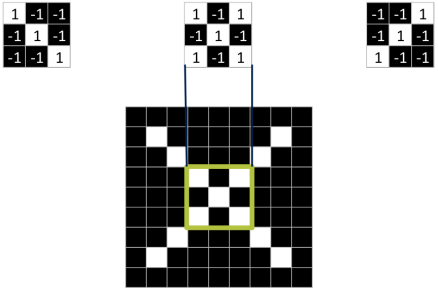
Filters Match Pieces of the Image



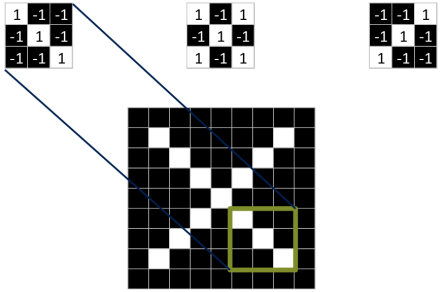
Filters Match Pieces of the Image



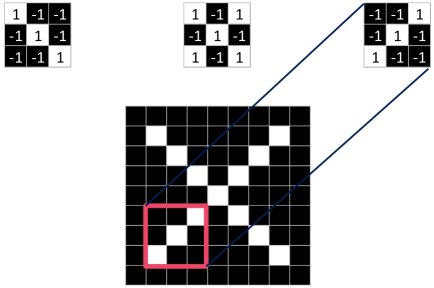
Filters Match Pieces of the Image



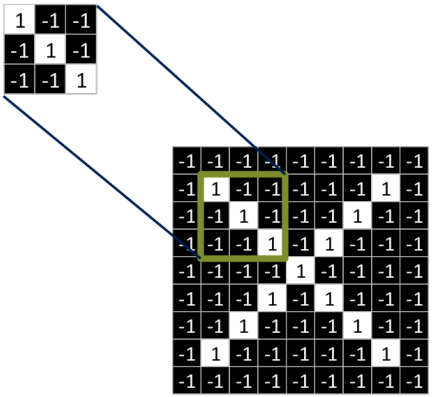
Filters Match Pieces of the Image



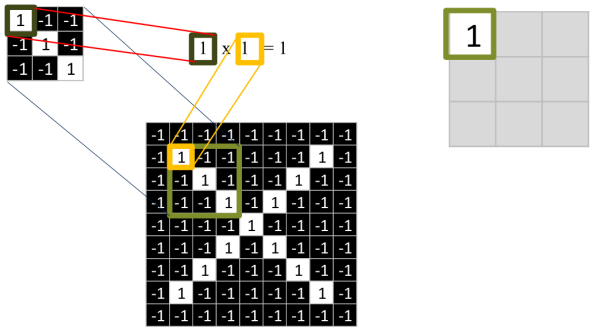
Filters Match Pieces of the Image



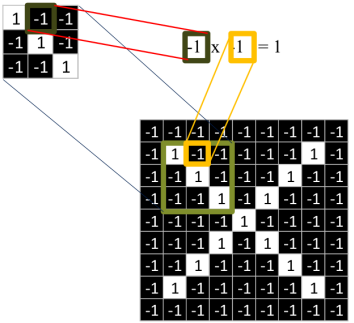
Filtering: The Math Behind the Match



Filtering: The Math Behind the Match

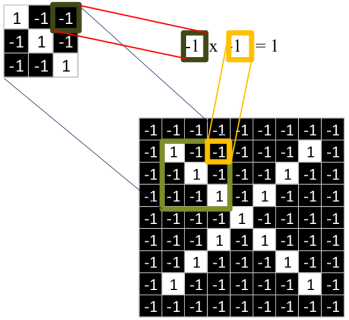


Filtering: The Math Behind the Match



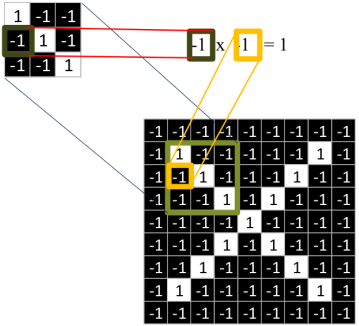


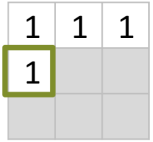
Filtering: The Math Behind the Match



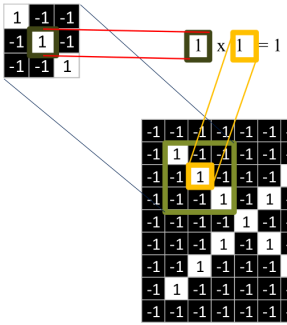


Filtering: The Math Behind the Match



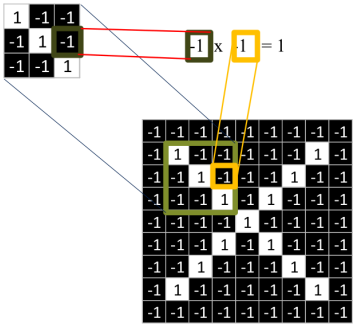


Filtering: The Math Behind the Match



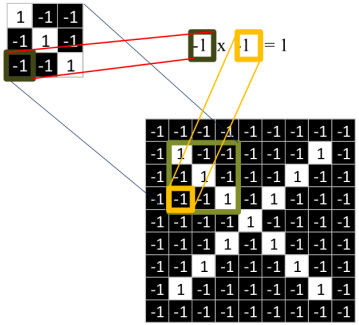


Filtering: The Math Behind the Match



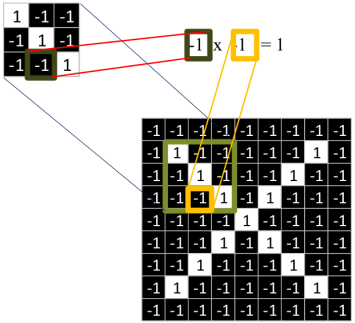
1	1	1
1	1	1

Filtering: The Math Behind the Match



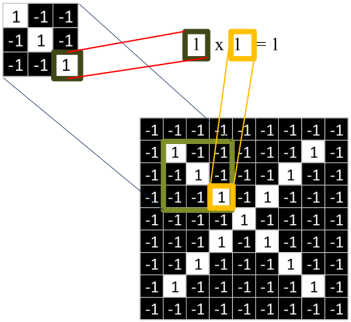
1	1	1
1	1	1
1		

Filtering: The Math Behind the Match



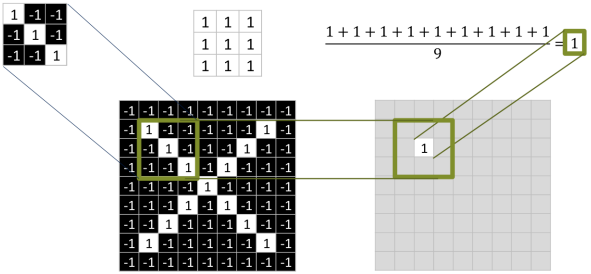
1	1	1
1	1	1
1	1	

Filtering: The Math Behind the Match

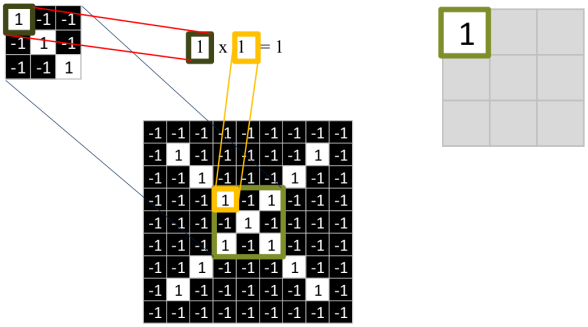


1	1	1
1	1	1
1	1	1

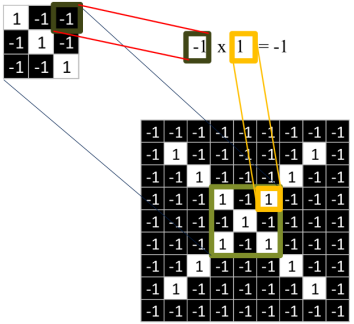
Filtering: The Math Behind the Match



Filtering: The Math Behind the Match

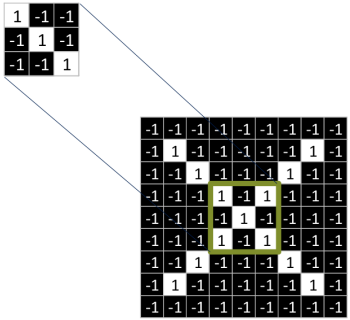


Filtering: The Math Behind the Match



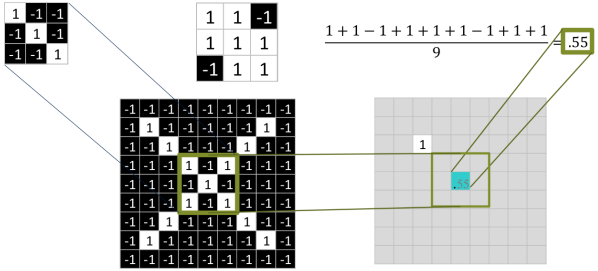
1	1	-1

Filtering: The Math Behind the Match

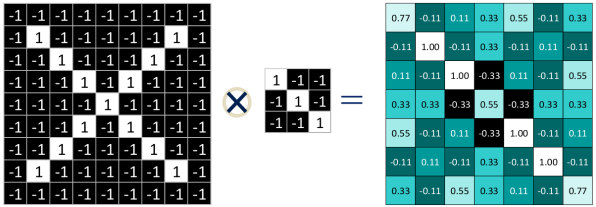


1	1	-1
1	1	1
-1	1	1

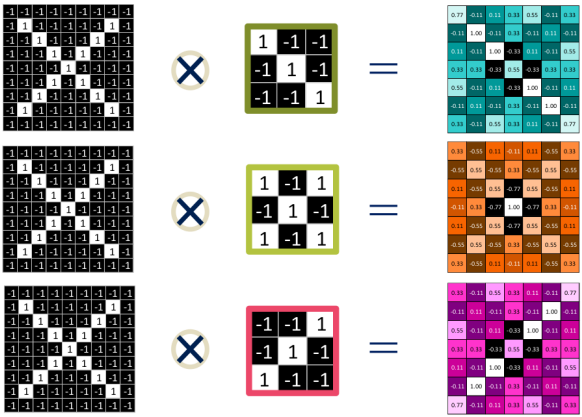
Filtering: The Math Behind the Match



Convolution: Trying Every Possible Match

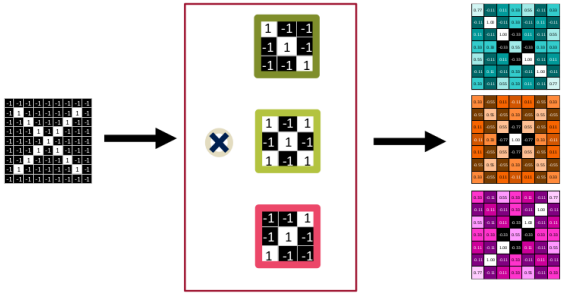


Three Filters Here, So Three Images Out

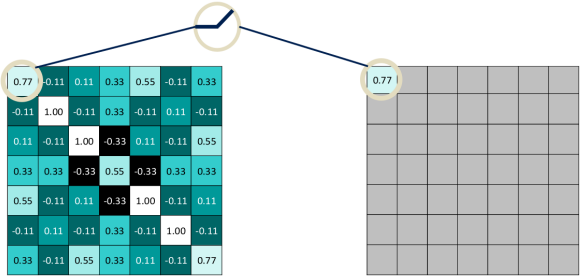


Convolution Layer

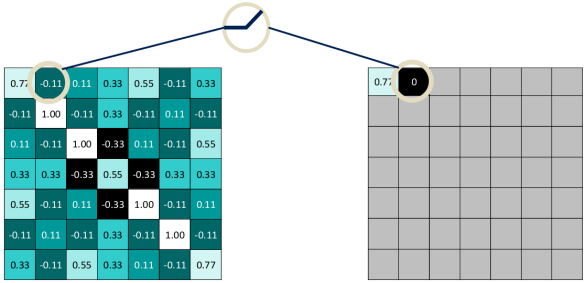
- One image becomes a stack of filtered images.



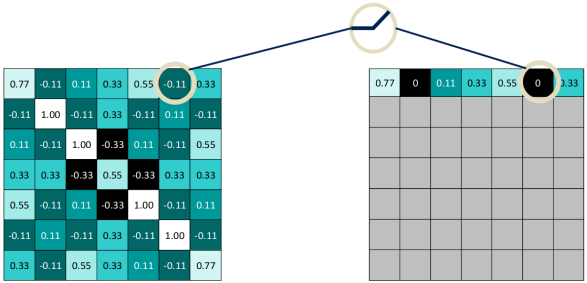
Rectified Linear Units (ReLUs)



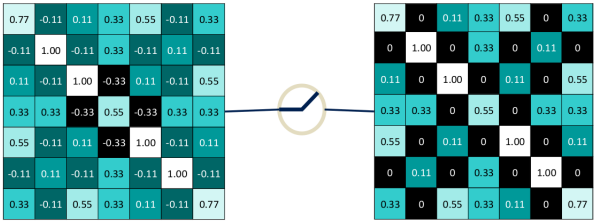
Rectified Linear Units (ReLUs)



Rectified Linear Units (ReLUs)

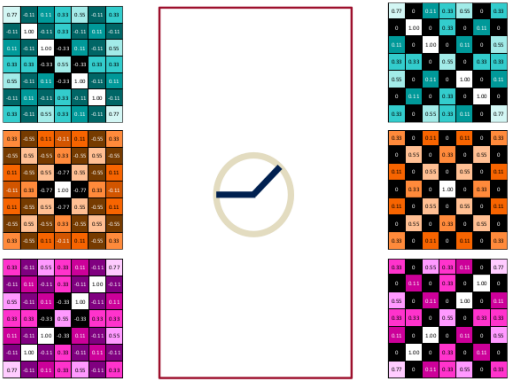


Rectified Linear Units (ReLUs)

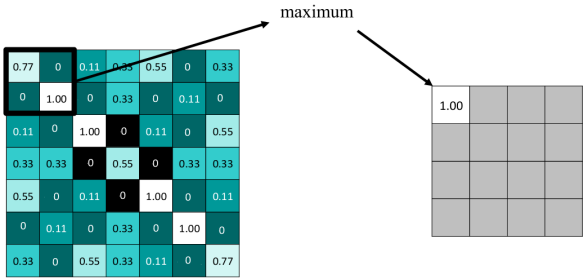


ReLU Layer

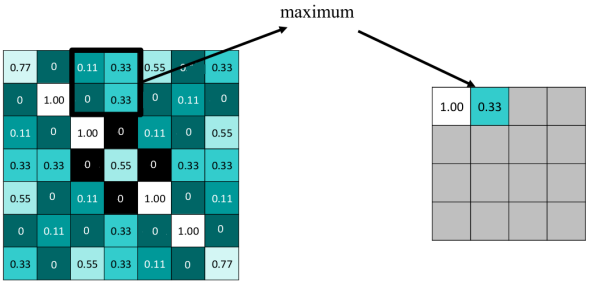
- A stack of images becomes a stack of images with no negative values.



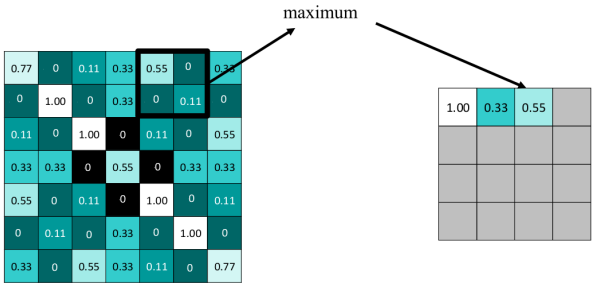
Pooling: Shrinking the Image Stack



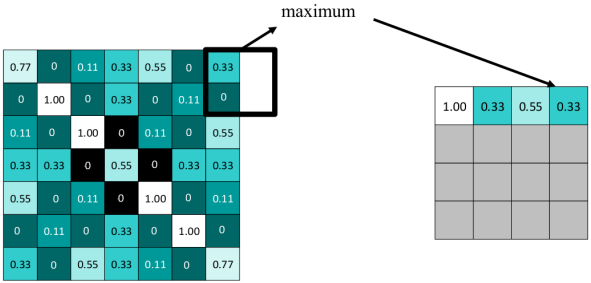
Pooling: Shrinking the Image Stack



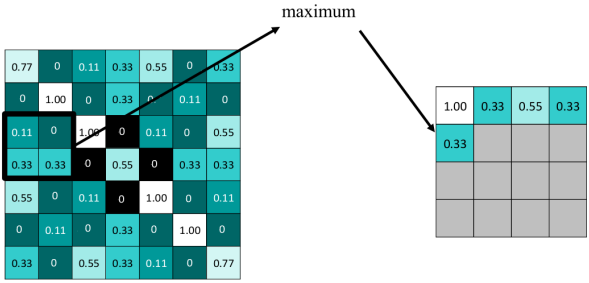
Pooling: Shrinking the Image Stack



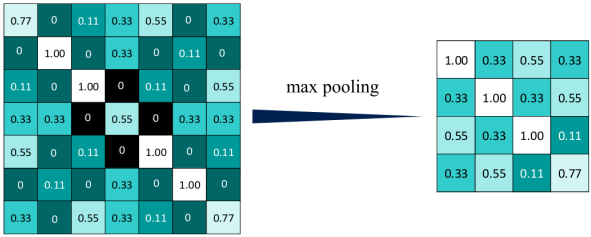
Pooling: Shrinking the Image Stack



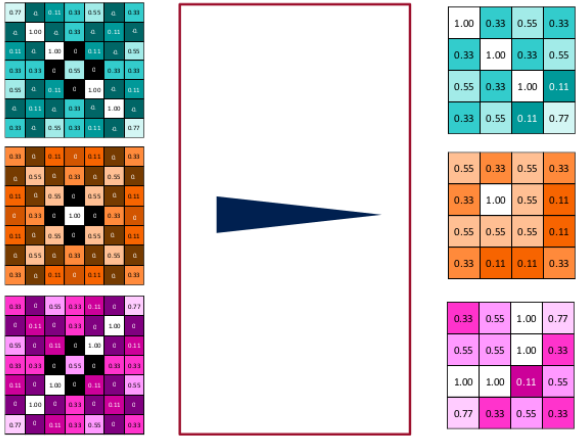
Pooling: Shrinking the Image Stack



Pooling: Shrinking the Image Stack

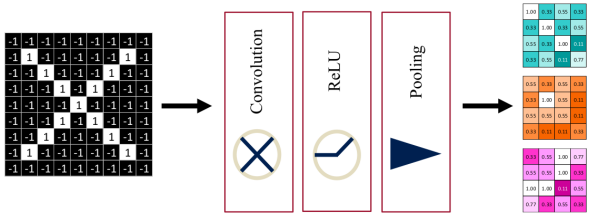


Repeat For All the Filtered Images

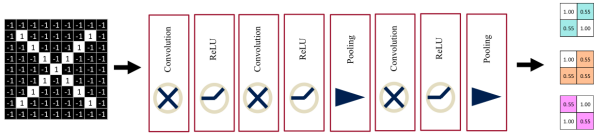


Layers Get Stacked

- The output of one becomes the input of the next.

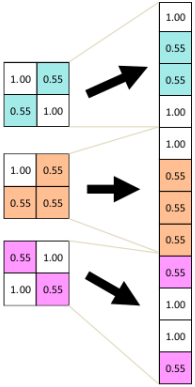


Deep Stacking



Fully Connected Layer

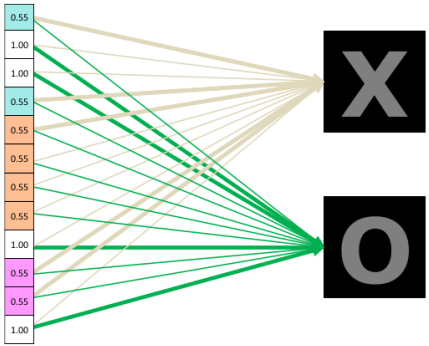
- Flattening the outputs before giving them to the fully connected layer.



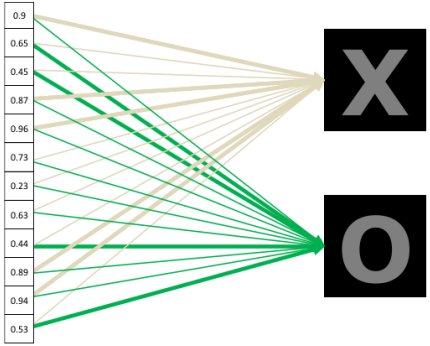
Fully Connected Layer



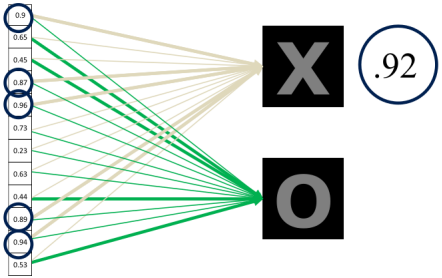
Fully Connected Layer



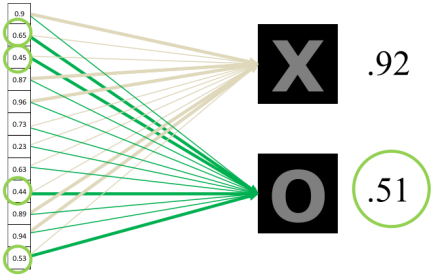
Fully Connected Layer



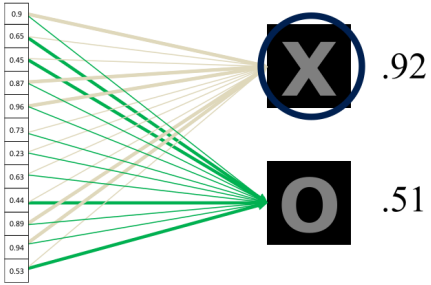
Fully Connected Layer



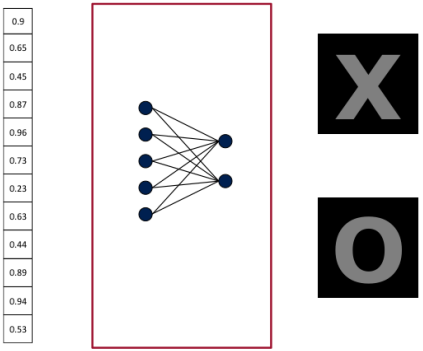
Fully Connected Layer



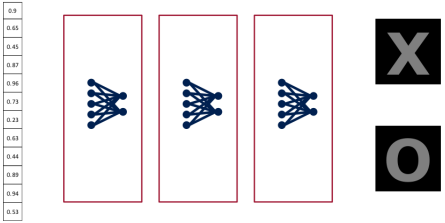
Fully Connected Layer



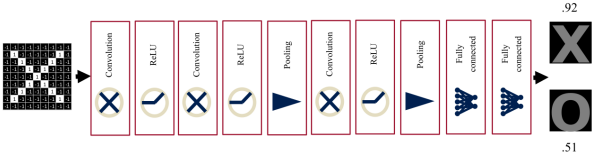
Fully Connected Layer



Fully Connected Layer



Putting It All Together




ENOUGH BLABLA!!
JUST SHOW ME THE CODE!!
imgflip.com



CNN in TensorFlow



CNN in TensorFlow (1/7)

- A CNN for the MNIST dataset with the following network.



CNN in TensorFlow (1/7)

- A CNN for the MNIST dataset with the following network.
- Conv. layer 1: computes 32 feature maps using a 5x5 filter with ReLU activation.



CNN in TensorFlow (1/7)

- A CNN for the MNIST dataset with the following network.
- Conv. layer 1: computes 32 feature maps using a 5x5 filter with ReLU activation.
- Pooling layer 1: max pooling layer with a 2x2 filter and stride of 2.



CNN in TensorFlow (1/7)

- A CNN for the MNIST dataset with the following network.
- Conv. layer 1: computes 32 feature maps using a 5x5 filter with ReLU activation.
- Pooling layer 1: max pooling layer with a 2x2 filter and stride of 2.
- Conv. layer 2: computes 64 feature maps using a 5x5 filter.



CNN in TensorFlow (1/7)

- A CNN for the MNIST dataset with the following network.
- Conv. layer 1: computes 32 feature maps using a 5x5 filter with ReLU activation.
- Pooling layer 1: max pooling layer with a 2x2 filter and stride of 2.
- Conv. layer 2: computes 64 feature maps using a 5x5 filter.
- Pooling layer 2: max pooling layer with a 2x2 filter and stride of 2.



CNN in TensorFlow (1/7)

- A CNN for the MNIST dataset with the following network.
- Conv. layer 1: computes 32 feature maps using a 5x5 filter with ReLU activation.
- Pooling layer 1: max pooling layer with a 2x2 filter and stride of 2.
- Conv. layer 2: computes 64 feature maps using a 5x5 filter.
- Pooling layer 2: max pooling layer with a 2x2 filter and stride of 2.
- Dense layer: densely connected layer with 1024 neurons.



CNN in TensorFlow (1/7)

- A CNN for the MNIST dataset with the following network.
- Conv. layer 1: computes 32 feature maps using a 5x5 filter with ReLU activation.
- Pooling layer 1: max pooling layer with a 2x2 filter and stride of 2.
- Conv. layer 2: computes 64 feature maps using a 5x5 filter.
- Pooling layer 2: max pooling layer with a 2x2 filter and stride of 2.
- Dense layer: densely connected layer with 1024 neurons.
- Softmax layer



CNN in TensorFlow (2/7)

- Conv. layer 1: computes 32 feature maps using a 5x5 filter with ReLU activation.
- Padding `same` is added to preserve width and height.



CNN in TensorFlow (2/7)

- Conv. layer 1: computes 32 feature maps using a 5x5 filter with ReLU activation.
- Padding `same` is added to preserve width and height.
- Input tensor shape: $[\texttt{batch_size}, 28, 28, 1]$



CNN in TensorFlow (2/7)

- Conv. layer 1: computes 32 feature maps using a 5x5 filter with ReLU activation.
- Padding `same` is added to preserve width and height.
- Input tensor shape: [`batch_size`, 28, 28, 1]
- Output tensor shape: [`batch_size`, 28, 28, 32]

```
# MNIST images are 28x28 pixels, and have one color channel: [28, 28, 1]

tf.keras.layers.Conv2D(kernel_size=5, filters=32, activation='relu', padding='same',
                       input_shape=[28, 28, 1])
```



CNN in TensorFlow (3/7)

- Pooling layer 1: max pooling layer with a 2x2 filter and stride of 2.



CNN in TensorFlow (3/7)

- Pooling layer 1: max pooling layer with a 2x2 filter and stride of 2.
- Input tensor shape: $[\texttt{batch_size}, 28, 28, 32]$



CNN in TensorFlow (3/7)

- Pooling layer 1: max pooling layer with a 2x2 filter and stride of 2.
- Input tensor shape: [batch_size, 28, 28, 32]
- Output tensor shape: [batch_size, 14, 14, 32]

```
tf.keras.layers.MaxPooling2D(pool_size=2, strides=2)
```



CNN in TensorFlow (4/7)

- Conv. layer 2: computes 64 feature maps using a 5x5 filter.
- Padding `same` is added to preserve width and height.



CNN in TensorFlow (4/7)

- Conv. layer 2: computes 64 feature maps using a 5x5 filter.
- Padding `same` is added to preserve width and height.
- Input tensor shape: $[\texttt{batch_size}, 14, 14, 32]$



CNN in TensorFlow (4/7)

- Conv. layer 2: computes 64 feature maps using a 5x5 filter.
- Padding `same` is added to preserve width and height.
- Input tensor shape: [`batch_size`, 14, 14, 32]
- Output tensor shape: [`batch_size`, 14, 14, 64]

```
tf.keras.layers.Conv2D(kernel_size=5, filters=64, activation='relu', padding='same')
```



CNN in TensorFlow (5/7)

- Pooling layer 2: max pooling layer with a 2x2 filter and stride of 2.



CNN in TensorFlow (5/7)

- Pooling layer 2: max pooling layer with a 2x2 filter and stride of 2.
- Input tensor shape: $[\texttt{batch_size}, 14, 14, 64]$



CNN in TensorFlow (5/7)

- Pooling layer 2: max pooling layer with a 2x2 filter and stride of 2.
- Input tensor shape: $[\texttt{batch_size}, 14, 14, 64]$
- Output tensor shape: $[\texttt{batch_size}, 7, 7, 64]$

```
tf.keras.layers.MaxPooling2D(pool_size=2, strides=2)
```



CNN in TensorFlow (6/7)

- Flatten tensor into a batch of vectors.



CNN in TensorFlow (6/7)

- Flatten tensor into a batch of vectors.
  - Input tensor shape: $[\texttt{batch_size}, 7, 7, 64]$



CNN in TensorFlow (6/7)

- Flatten tensor into a batch of vectors.
  - Input tensor shape: $[\texttt{batch_size}, 7, 7, 64]$
  - Output tensor shape: $[\texttt{batch_size}, 7 \times 7 \times 64]$

```
tf.keras.layers.Flatten()
```



CNN in TensorFlow (6/7)

- Flatten tensor into a batch of vectors.
  - Input tensor shape: $[\texttt{batch_size}, 7, 7, 64]$
  - Output tensor shape: $[\texttt{batch_size}, 7 \times 7 \times 64]$

```
tf.keras.layers.Flatten()
```

- Dense layer: densely connected layer with 1024 neurons.



CNN in TensorFlow (6/7)

- Flatten tensor into a batch of vectors.
  - Input tensor shape: $[\texttt{batch_size}, 7, 7, 64]$
  - Output tensor shape: $[\texttt{batch_size}, 7 \times 7 \times 64]$

```
tf.keras.layers.Flatten()
```

- Dense layer: densely connected layer with 1024 neurons.
  - Input tensor shape: $[\texttt{batch_size}, 7 \times 7 \times 64]$



CNN in TensorFlow (6/7)

- Flatten tensor into a batch of vectors.
  - Input tensor shape: $[\texttt{batch_size}, 7, 7, 64]$
  - Output tensor shape: $[\texttt{batch_size}, 7 \times 7 \times 64]$

```
tf.keras.layers.Flatten()
```

- Dense layer: densely connected layer with 1024 neurons.
  - Input tensor shape: $[\texttt{batch_size}, 7 \times 7 \times 64]$
  - Output tensor shape: $[\texttt{batch_size}, 1024]$

```
tf.keras.layers.Dense(1024, activation='relu')
```



CNN in TensorFlow (6/7)

- Flatten tensor into a batch of vectors.
  - Input tensor shape: $[\texttt{batch_size}, 7, 7, 64]$
  - Output tensor shape: $[\texttt{batch_size}, 7 \times 7 \times 64]$

```
tf.keras.layers.Flatten()
```

- Dense layer: densely connected layer with 1024 neurons.
  - Input tensor shape: $[\texttt{batch_size}, 7 \times 7 \times 64]$
  - Output tensor shape: $[\texttt{batch_size}, 1024]$

```
tf.keras.layers.Dense(1024, activation='relu')
```

- Softmax layer: softmax layer with 10 neurons.

CNN in TensorFlow (6/7)

- Flatten tensor into a batch of vectors.
  - Input tensor shape: $[\texttt{batch_size}, 7, 7, 64]$
  - Output tensor shape: $[\texttt{batch_size}, 7 \times 7 \times 64]$

```
tf.keras.layers.Flatten()
```

- Dense layer: densely connected layer with 1024 neurons.
  - Input tensor shape: $[\texttt{batch_size}, 7 \times 7 \times 64]$
  - Output tensor shape: $[\texttt{batch_size}, 1024]$

```
tf.keras.layers.Dense(1024, activation='relu')
```

- Softmax layer: softmax layer with 10 neurons.
  - Input tensor shape: $[\texttt{batch_size}, 1024]$



CNN in TensorFlow (6/7)

- Flatten tensor into a batch of vectors.
  - Input tensor shape: $[\texttt{batch_size}, 7, 7, 64]$
  - Output tensor shape: $[\texttt{batch_size}, 7 \times 7 \times 64]$

```
tf.keras.layers.Flatten()
```

- Dense layer: densely connected layer with 1024 neurons.
  - Input tensor shape: $[\texttt{batch_size}, 7 \times 7 \times 64]$
  - Output tensor shape: $[\texttt{batch_size}, 1024]$

```
tf.keras.layers.Dense(1024, activation='relu')
```

- Softmax layer: softmax layer with 10 neurons.
  - Input tensor shape: $[\texttt{batch_size}, 1024]$
  - Output tensor shape: $[\texttt{batch_size}, 10]$

```
tf.keras.layers.Dense(10, activation='softmax')
```

CNN in TensorFlow (7/7)

```
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(kernel_size=5, filters=32, activation='relu', padding='same',
                           input_shape=[28, 28, 1]),
    tf.keras.layers.MaxPooling2D(pool_size=2, strides=2),
    tf.keras.layers.Conv2D(kernel_size=5, filters=64, activation='relu', padding='same'),
    tf.keras.layers.MaxPooling2D(pool_size=2, strides=2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(1024, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
  ])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```





Training CNNs

Training CNN (1/4)

- Let's see how to use backpropagation on a single convolutional layer.



Training CNN (1/4)

- Let's see how to use backpropagation on a single convolutional layer.
- Assume we have an input `X` of size 3x3 and a single filter `W` of size 2x2.



Training CNN (1/4)

- Let's see how to use backpropagation on a single convolutional layer.
- Assume we have an input `X` of size 3x3 and a single filter `W` of size 2x2.
- No padding and stride $= 1$.



Training CNN (1/4)

- Let's see how to use backpropagation on a single convolutional layer.
- Assume we have an input `X` of size 3x3 and a single filter `W` of size 2x2.
- No padding and stride $= 1$.
- It generates an output `H` of size 2x2.





Training CNN (2/4)

- Forward pass

Training CNN (2/4)

- Forward pass



$$h_{11} = W_{11}X_{11} + W_{12}X_{12} + W_{21}X_{21} + W_{22}X_{22}$$

Training CNN (2/4)

- Forward pass



$$h_{11} = W_{11}X_{11} + W_{12}X_{12} + W_{21}X_{21} + W_{22}X_{22}$$

$$h_{12} = W_{11}X_{12} + W_{12}X_{13} + W_{21}X_{22} + W_{22}X_{23}$$

Training CNN (2/4)

- Forward pass



$$h_{11} = W_{11}X_{11} + W_{12}X_{12} + W_{21}X_{21} + W_{22}X_{22}$$

$$h_{12} = W_{11}X_{12} + W_{12}X_{13} + W_{21}X_{22} + W_{22}X_{23}$$

$$h_{21} = W_{11}X_{21} + W_{12}X_{22} + W_{21}X_{31} + W_{22}X_{32}$$

Training CNN (2/4)

- Forward pass



$$h_{11} = W_{11}X_{11} + W_{12}X_{12} + W_{21}X_{21} + W_{22}X_{22}$$

$$h_{12} = W_{11}X_{12} + W_{12}X_{13} + W_{21}X_{22} + W_{22}X_{23}$$

$$h_{21} = W_{11}X_{21} + W_{12}X_{22} + W_{21}X_{31} + W_{22}X_{32}$$

$$h_{22} = W_{11}X_{22} + W_{12}X_{23} + W_{21}X_{32} + W_{22}X_{33}$$

Training CNN (3/4)

- Backward pass
- E is the error: $\mathrm{E} = \mathrm{E}_{\mathrm{h}_{11}} + \mathrm{E}_{\mathrm{h}_{12}} + \mathrm{E}_{\mathrm{h}_{21}} + \mathrm{E}_{\mathrm{h}_{22}}$



Training CNN (3/4)

- Backward pass
- E is the error: $\mathrm{E} = \mathrm{E}_{\mathrm{h}_{11}} + \mathrm{E}_{\mathrm{h}_{12}} + \mathrm{E}_{\mathrm{h}_{21}} + \mathrm{E}_{\mathrm{h}_{22}}$



$$\frac{\partial \mathrm{E}}{\partial \mathrm{W}_{11}} = \frac{\partial \mathrm{E}_{\mathrm{h}_{11}}}{\partial \mathrm{h}_{11}} \frac{\partial \mathrm{h}_{11}}{\partial \mathrm{W}_{11}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{12}}}{\partial \mathrm{h}_{12}} \frac{\partial \mathrm{h}_{12}}{\partial \mathrm{W}_{11}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{21}}}{\partial \mathrm{h}_{21}} \frac{\partial \mathrm{h}_{21}}{\partial \mathrm{W}_{11}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{22}}}{\partial \mathrm{h}_{22}} \frac{\partial \mathrm{h}_{22}}{\partial \mathrm{W}_{11}}$$

Training CNN (3/4)

- Backward pass
- E is the error: $\mathrm{E} = \mathrm{E}_{\mathrm{h}_{11}} + \mathrm{E}_{\mathrm{h}_{12}} + \mathrm{E}_{\mathrm{h}_{21}} + \mathrm{E}_{\mathrm{h}_{22}}$



$$\frac{\partial \mathrm{E}}{\partial \mathrm{W}_{11}} = \frac{\partial \mathrm{E}_{\mathrm{h}_{11}}}{\partial \mathrm{h}_{11}} \frac{\partial \mathrm{h}_{11}}{\partial \mathrm{W}_{11}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{12}}}{\partial \mathrm{h}_{12}} \frac{\partial \mathrm{h}_{12}}{\partial \mathrm{W}_{11}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{21}}}{\partial \mathrm{h}_{21}} \frac{\partial \mathrm{h}_{21}}{\partial \mathrm{W}_{11}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{22}}}{\partial \mathrm{h}_{22}} \frac{\partial \mathrm{h}_{22}}{\partial \mathrm{W}_{11}}$$

$$\frac{\partial \mathrm{E}}{\partial \mathrm{W}_{12}} = \frac{\partial \mathrm{E}_{\mathrm{h}_{11}}}{\partial \mathrm{h}_{11}} \frac{\partial \mathrm{h}_{11}}{\partial \mathrm{W}_{12}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{12}}}{\partial \mathrm{h}_{12}} \frac{\partial \mathrm{h}_{12}}{\partial \mathrm{W}_{12}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{21}}}{\partial \mathrm{h}_{21}} \frac{\partial \mathrm{h}_{21}}{\partial \mathrm{W}_{12}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{22}}}{\partial \mathrm{h}_{22}} \frac{\partial \mathrm{h}_{22}}{\partial \mathrm{W}_{12}}$$

Training CNN (3/4)

- Backward pass
- E is the error: $\mathrm{E} = \mathrm{E}_{\mathrm{h}_{11}} + \mathrm{E}_{\mathrm{h}_{12}} + \mathrm{E}_{\mathrm{h}_{21}} + \mathrm{E}_{\mathrm{h}_{22}}$



$$\frac{\partial \mathrm{E}}{\partial \mathrm{W}_{11}} = \frac{\partial \mathrm{E}_{\mathrm{h}_{11}}}{\partial \mathrm{h}_{11}} \frac{\partial \mathrm{h}_{11}}{\partial \mathrm{W}_{11}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{12}}}{\partial \mathrm{h}_{12}} \frac{\partial \mathrm{h}_{12}}{\partial \mathrm{W}_{11}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{21}}}{\partial \mathrm{h}_{21}} \frac{\partial \mathrm{h}_{21}}{\partial \mathrm{W}_{11}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{22}}}{\partial \mathrm{h}_{22}} \frac{\partial \mathrm{h}_{22}}{\partial \mathrm{W}_{11}}$$

$$\frac{\partial \mathrm{E}}{\partial \mathrm{W}_{12}} = \frac{\partial \mathrm{E}_{\mathrm{h}_{11}}}{\partial \mathrm{h}_{11}} \frac{\partial \mathrm{h}_{11}}{\partial \mathrm{W}_{12}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{12}}}{\partial \mathrm{h}_{12}} \frac{\partial \mathrm{h}_{12}}{\partial \mathrm{W}_{12}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{21}}}{\partial \mathrm{h}_{21}} \frac{\partial \mathrm{h}_{21}}{\partial \mathrm{W}_{12}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{22}}}{\partial \mathrm{h}_{22}} \frac{\partial \mathrm{h}_{22}}{\partial \mathrm{W}_{12}}$$

$$\frac{\partial \mathrm{E}}{\partial \mathrm{W}_{21}} = \frac{\partial \mathrm{E}_{\mathrm{h}_{11}}}{\partial \mathrm{h}_{11}} \frac{\partial \mathrm{h}_{11}}{\partial \mathrm{W}_{21}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{12}}}{\partial \mathrm{h}_{12}} \frac{\partial \mathrm{h}_{12}}{\partial \mathrm{W}_{21}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{21}}}{\partial \mathrm{h}_{21}} \frac{\partial \mathrm{h}_{21}}{\partial \mathrm{W}_{21}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{22}}}{\partial \mathrm{h}_{22}} \frac{\partial \mathrm{h}_{22}}{\partial \mathrm{W}_{21}}$$

Training CNN (3/4)

- Backward pass
- E is the error: $\mathrm{E} = \mathrm{E}_{\mathrm{h}_{11}} + \mathrm{E}_{\mathrm{h}_{12}} + \mathrm{E}_{\mathrm{h}_{21}} + \mathrm{E}_{\mathrm{h}_{22}}$



$$\frac{\partial \mathrm{E}}{\partial \mathrm{W}_{11}} = \frac{\partial \mathrm{E}_{\mathrm{h}_{11}}}{\partial \mathrm{h}_{11}} \frac{\partial \mathrm{h}_{11}}{\partial \mathrm{W}_{11}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{12}}}{\partial \mathrm{h}_{12}} \frac{\partial \mathrm{h}_{12}}{\partial \mathrm{W}_{11}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{21}}}{\partial \mathrm{h}_{21}} \frac{\partial \mathrm{h}_{21}}{\partial \mathrm{W}_{11}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{22}}}{\partial \mathrm{h}_{22}} \frac{\partial \mathrm{h}_{22}}{\partial \mathrm{W}_{11}}$$

$$\frac{\partial \mathrm{E}}{\partial \mathrm{W}_{12}} = \frac{\partial \mathrm{E}_{\mathrm{h}_{11}}}{\partial \mathrm{h}_{11}} \frac{\partial \mathrm{h}_{11}}{\partial \mathrm{W}_{12}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{12}}}{\partial \mathrm{h}_{12}} \frac{\partial \mathrm{h}_{12}}{\partial \mathrm{W}_{12}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{21}}}{\partial \mathrm{h}_{21}} \frac{\partial \mathrm{h}_{21}}{\partial \mathrm{W}_{12}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{22}}}{\partial \mathrm{h}_{22}} \frac{\partial \mathrm{h}_{22}}{\partial \mathrm{W}_{12}}$$

$$\frac{\partial \mathrm{E}}{\partial \mathrm{W}_{21}} = \frac{\partial \mathrm{E}_{\mathrm{h}_{11}}}{\partial \mathrm{h}_{11}} \frac{\partial \mathrm{h}_{11}}{\partial \mathrm{W}_{21}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{12}}}{\partial \mathrm{h}_{12}} \frac{\partial \mathrm{h}_{12}}{\partial \mathrm{W}_{21}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{21}}}{\partial \mathrm{h}_{21}} \frac{\partial \mathrm{h}_{21}}{\partial \mathrm{W}_{21}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{22}}}{\partial \mathrm{h}_{22}} \frac{\partial \mathrm{h}_{22}}{\partial \mathrm{W}_{21}}$$

$$\frac{\partial \mathrm{E}}{\partial \mathrm{W}_{22}} = \frac{\partial \mathrm{E}_{\mathrm{h}_{11}}}{\partial \mathrm{h}_{11}} \frac{\partial \mathrm{h}_{11}}{\partial \mathrm{W}_{22}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{12}}}{\partial \mathrm{h}_{12}} \frac{\partial \mathrm{h}_{12}}{\partial \mathrm{W}_{22}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{21}}}{\partial \mathrm{h}_{21}} \frac{\partial \mathrm{h}_{21}}{\partial \mathrm{W}_{22}} + \frac{\partial \mathrm{E}_{\mathrm{h}_{22}}}{\partial \mathrm{h}_{22}} \frac{\partial \mathrm{h}_{22}}{\partial \mathrm{W}_{22}}$$

Training CNN (4/4)

- Update the wights W



$$W_{11}^{(next)} = W_{11} - \eta \frac{\partial E}{\partial W_{11}}$$
$$W_{12}^{(next)} = W_{12} - \eta \frac{\partial E}{\partial W_{12}}$$
$$W_{21}^{(next)} = W_{21} - \eta \frac{\partial E}{\partial W_{21}}$$
$$W_{22}^{(next)} = W_{22} - \eta \frac{\partial E}{\partial W_{22}}$$

Summary



Summary

- Receptive fields and filters
- Convolution operation
- Padding and strides
- Pooling layer
- Flattening, dropout, dense



Reference

- Tensorflow and Deep Learning without a PhD
  `https://codelabs.developers.google.com/codelabs/cloud-tensorflow-mnist`
- Hands-on machine learning with Scikit-Learn and TensorFlow (Ch. 14)



Questions?